- The metric used: sum of weights instead of path lengths;
- Consequently, the container that needs to be used to keep track of the next vertices to be visited: we can't make do with a plain queue anymore, we need a priority queue.

Everything else, the logic of the algorithm and the auxiliary data used, are similar to BFS: that's very convenient for us, because we can re-write this algorithm from listing 14.2 with minimal changes. If you think that this similarity is a coincidence, though, hold your breath till section 14.5[8].

14.4.2 Implementation

Listing 14.6 describes Dijkstra's algorithm in detail: comparing it to listing 14.2, you can see how it resembles BFS algorithm, so much so that we can use the same algorithm shown in listing 14.3 to reconstruct the shortest path for Dijkstra's as well. Nonetheless, we need to be even more careful about performance, in this case.

Listing 14.12 The Dijkstra algorithm

```
function dijkstra(graph, start,
isGoal)                                          #1
  queue ← new
PriorityQueue()
#2
  queue.insert(start,
0)                                                        #3
  distances[v] ← inf (" v ∈ graph | v <>
start)                                #4
```